

Design and Implementation of markTeX: An Extensible Markdown-to-LaTeX Compiler in Haskell

Aditya Patel

Himavanth Karpurapu

May 2026

Abstract

*This report describes **markTeX**, a small domain-specific language and accompanying compiler that translates a Markdown-inspired source format into LaTeX. The system is implemented in Haskell using the Parsec parser-combinator library and follows a conventional compiler pipeline composed of parsing, abstract syntax tree construction, semantic validation, and target code generation. The language preserves the readability of Markdown while adding a focused set of LaTeX-oriented features, including labeled headers, internal cross-references, citations, recursive inline formatting, and verbatim code blocks. The implementation prioritizes correctness over feature breadth, and rejects documents containing duplicate labels or undefined references before any LaTeX output is produced. The resulting compiler accepts a single source file, emits a well-formed `.tex` document, and invokes the standard `pdflatex` and `bibtex` toolchain to produce a PDF.*

1 Introduction

Markdown has become a default authoring format for technical documentation, blog posts, README files, and informal reports because its syntax is lightweight, easy to read in source form, and easy to type. The cost of that simplicity, however, is expressiveness. Standard Markdown lacks first-class support for citations, labeled sections, cross-references, and the structural conventions that academic and technical writing depends on. LaTeX provides those features in depth, but its source is verbose, visually noisy, and demands a level of attention to syntax

that pulls the author away from content.

markTeX sits between these two systems. It accepts a source format that looks and feels like Markdown for the constructs that Markdown handles well, and adds a small set of LaTeX-oriented extensions for the constructs that Markdown does not. In **markTeX**, a labeled section is written as `## Features {#sec:features}`, and a citation is written as `[@paper1, paper2]`. The equivalent LaTeX requires `\subsection{Features} \label{sec:features}` and `\cite{paper1,paper2}` respectively. The compiler emits the corresponding LaTeX automatically.

The project is implementation-oriented. Its purpose is to apply a set of ideas from programming language design, including grammar definition, parser construction, abstract syntax tree representation, semantic analysis, and code generation, to a problem domain where those ideas produce a usable tool. The sections that follow describe the language, the architecture, the data representations, and the design decisions that shaped the final system.

2 Motivation

The motivation for **markTeX** comes from a recurring tension in technical writing. Authors want to focus on content. Publishers and academic venues require structure, citations, and cross-references. Markdown serves the first goal and LaTeX serves the second, but neither serves both comfortably.

Consider a short paragraph that introduces a section, references another section, and cites two papers. In LaTeX, the author writes the following:

Listing 1: Example of LaTeX source for a short reference-heavy paragraph.

```
\section{Introduction}\label{sec:intro}

See \hyperref[sec:features]{the features
  section} (\Cref{sec:features}).
\cite{paper1,paper_2026}
```

In markTeX, the same content is written as follows:

Listing 2: Equivalent markTeX source.

```
# Introduction {#sec:intro}

See [the features section] (#sec:features).
[@paper1, paper_2026]
```

The markTeX version is shorter, contains fewer escape characters, and reads closer to natural text. The compiler is then responsible for producing the LaTeX output, validating that `sec:features` actually exists somewhere in the document, and applying any escaping that the target language requires. This separation of authoring concerns from formatting concerns is the central motivating idea of the project.

3 Project Goals & Scope

The project pursues a deliberately bounded set of goals. The intention is to demonstrate the full compiler pipeline on a language large enough to be interesting but small enough to implement carefully. Specifically, the system aims to:

- Define a small Markdown-inspired surface language with selected LaTeX-oriented extensions.
- Parse the surface language into a structured abstract syntax tree using Parsec parser combinators.
- Support recursive inline formatting, so that constructs such as bold text containing italic text containing underline text are represented and rendered correctly.
- Support optional document metadata for title, author, and date.

- Support labeled headers, single-target references, and multi-target references.
- Support citations with one or more citation keys per citation node.
- Validate the document by detecting duplicate labels and undefined references before any output is generated.
- Emit a complete LaTeX document and invoke `pdflatex` and `bibtex` to produce a PDF.

Full Markdown compatibility, tables, images, ordered lists, and mathematical expressions are explicitly out of scope. These omissions are discussed further in Section 15.

4 Language Design

The markTeX language is organized around three layers: optional document-level metadata, block-level constructs that define the document structure, and inline constructs that define formatting and semantic annotations inside blocks. Each layer corresponds to a separate group of parsers in the implementation, which keeps the grammar modular and easy to extend.

4.1 Document Metadata

Documents may begin with an optional metadata block. Each metadata line starts with a percent sign, names a field, and provides a value. The compiler currently recognizes three fields: title, author, and date.

Listing 3: Document metadata in markTeX.

```
% Title: My ~~Complex~~ **_Simple*_ Paper
**
% Author: \~John\_Doe
% Date: 2026
```

Metadata values are parsed using the same inline parser used in the body of the document, which means that a title can contain bold text, italic text,

underline text, strikethrough text, and escaped LaTeX characters. The compiler emits the corresponding `\title`, `\author`, and `\date` commands when at least one metadata field is present, and inserts a `\maketitle` at the start of the body.

4.2 Inline Constructs

Inline constructs are the building blocks of paragraph content. The supported inline forms and their LaTeX mappings appear in Table 1.

A non-trivial requirement of the design is that inline constructs nest. The following example, which appears in the reference test document, is valid `markTeX`:

Listing 4: Nested inline formatting from the test document.

```
**start bold with *italic and _underline
    and ~~strike~~ inside_ inside*.
```

The grammar must therefore allow the contents of a bold node to contain italic, underline, strike, and any other inline construct, including further bold nodes. This is reflected in the AST, which represents each formatting constructor as carrying a list of inline values rather than a flat string.

4.3 Block-Level Constructs

Block-level constructs define the structural skeleton of the document. The implementation supports four block types: paragraphs, headers, unordered lists, and code blocks. Headers may optionally carry a label, written in braces immediately after the header text:

Listing 5: Labeled section syntax.

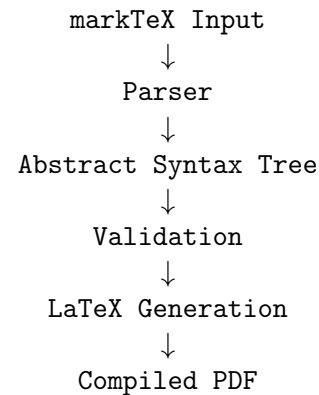
```
## Features {#sec:features}
```

Unordered lists are introduced by lines beginning with a hyphen. Each item may contain its own block-level content, including nested paragraphs and nested headers, provided that the nested content is indented by two spaces. This allows authoring patterns such as embedding a labeled subsection inside a list item, which the test document exercises.

Code blocks are delimited by triple backticks. They are treated as raw, opaque text. None of the inline formatting rules apply inside a code block, which means that the sequence `**not bold**` appears in the rendered PDF exactly as written, rather than being interpreted as a bold node. This behavior matches user expectations for code examples and avoids surprising interactions between the host language syntax and the content of the code block.

5 System Architecture

The compiler is organized as a four-stage pipeline. Each stage has a narrowly scoped responsibility, and each consumes the output of the previous stage.



The parser converts the source text into a structured AST. The validation stage walks the AST twice, first to collect every defined label and detect duplicates, and then to confirm that every reference points to one of the collected labels. The renderer assumes a valid AST and emits LaTeX source. The final stage invokes `pdflatex` and `bibtex` from the host operating system to produce a PDF.

This separation has practical consequences for the implementation. Because validation runs before rendering, the renderer never has to consider whether a reference target exists. Because parsing produces a typed AST rather than raw strings, the renderer never has to re-interpret the source. Each stage operates on a representation that is well suited to the work it performs.

Feature	markTeX Syntax	LaTeX Output
Italic	<code>*text*</code>	<code>\textit{text}</code>
Bold	<code>**text**</code>	<code>\textbf{text}</code>
Underline	<code>_text_</code>	<code>\underline{text}</code>
Strikethrough	<code>~~text~~</code>	<code>\sout{text}</code>
Citation	<code>[@paper1, paper2]</code>	<code>\cite{paper1,paper2}</code>
Single-label reference	<code>[text] (#sec:intro)</code>	<code>\hyperref[sec:intro]{text} (\Cref{sec:intro})</code>
Empty-text reference	<code>[] (#sec:intro)</code>	<code>\Cref{sec:intro}</code>
Multi-label reference	<code>[text] (#a,b,c)</code>	<code>\hyperref[a]{text} (\Cref{a,b,c})</code>

Table 1: Supported inline syntax in markTeX and the corresponding LaTeX output.

6 Implementation & Module Structure

The implementation is written in Haskell and uses Parsec for parsing. Haskell suits the project well because algebraic data types map directly onto the recursive structure of a document. Pattern matching, which is the natural way to walk algebraic data in Haskell, makes the validation and rendering stages short and easy to read.

The source is organized into four modules:

- `MarkTex.AST` defines the `Document`, `Block`, and `Inline` types, along with the `Label` and `CitationKey` aliases.
- `MarkTex.Parser` defines the Parsec parsers for metadata, blocks, inline constructs, labels, citations, and references, and exposes a public `parseDocument` entry point.
- `MarkTex.Render.LaTeX` defines label collection, reference validation, and the rendering functions that convert AST nodes into LaTeX strings.
- `Main` reads command-line arguments, drives the parse, runs validation and rendering, writes the resulting `.tex` file, and invokes the LaTeX toolchain.

6.1 Use of Parsec

Parsec is a parser-combinator library. Rather than describing the grammar in a separate generator file, the grammar is expressed as ordinary Haskell functions that compose into larger functions. Small

parsers for individual tokens, such as a single citation key or a single label, are combined into parsers for full constructs, such as a header or a reference.

The top-level document parser illustrates this pattern. It runs the metadata parser, then repeatedly runs the block parser until end-of-file, and finally packages the results into a `Document` value:

Listing 6: Top-level document parser (from `MarkTex.Parser`).

```
documentParser :: GenParser Char st
  Document
documentParser = do
  many newline
  (title, author, date) <- metaParser
  blocks <- many blockParser
  many newline
  eof
  return $ Document title author date
    blocks
```

The block parser is itself a small dispatcher built from the alternative combinator. It tries each block type in order, using `try` so that a failed attempt does not consume input:

Listing 7: Block parser dispatch (from `MarkTex.Parser`).

```
blockParser :: GenParser Char st Block
blockParser =
  try headerParser
<|> try listParser
<|> try codeParser
<|> try paragraphParser
<?> "block"
```

The header parser reads one or more `#` characters

to determine the nesting depth, parses the inline content, and then attempts to parse an optional label. The label is wrapped in `Maybe`, which tracks about whether a label is actually present:

Listing 8: Header parser (from `MarkTex.Parser`).

```
headerParser :: GenParser Char st Block
headerParser = do
  level <- length <$> many1 (char '#')
  hspaces
  inlines <- many1 inlineParser
  mLabel <- optionMaybe labelParser
  optional (many1 newline)
  return $ Header level inlines mLabel
```

6.2 Parsing Inline Syntax

Inline parsing is the most intricate part of the implementation because inline constructs nest, and several of them share prefix characters. Bold and italic both begin with an asterisk. Citations and references both begin with a square bracket. The parser distinguishes these cases through ordering and through small lookahead helpers.

The bold parser, for example, must avoid being triggered by italic delimiters, and the italic parser must avoid being triggered by bold delimiters. The implementation handles this by giving each formatting parser a dedicated end-of-construct predicate:

Listing 9: Bold and italic delimiter handling (from `MarkTex.Parser`).

```
boldParser :: GenParser Char st Inline
boldParser = do
  string "**"
  inlines <- manyTill inlineParser boldEnd
  return $ Bold inlines

boldEnd :: GenParser Char st String
boldEnd = try $ do
  notFollowedBy (string "***")
  string "**"

italicParser :: GenParser Char st Inline
italicParser = do
  char '*'
  inlines <- manyTill inlineParser
    italicEnd
  return $ Italic inlines
```

Citations and references are also distinguished early in the parse. The citation parser commits to the construct after seeing the `[@` prefix, while the reference parser handles the unprefix `[...](#...)` form. Both produce inline AST nodes rather than block-level nodes, so they can appear anywhere inline content is permitted, including inside bold text, italic text, or list items.

A small but important detail is the escaped-character parser, which handles sequences such as `\#` and `\_`. These sequences allow authors to write text that would otherwise be interpreted as syntax, and they are particularly useful in metadata fields and in paragraphs that describe LaTeX itself.

7 Abstract Syntax Tree Design

The AST is the central intermediate representation of the compiler. It is defined in `MarkTex.AST` and consists of three primary types plus two aliases:

Listing 10: AST definitions (from `MarkTex.AST`).

```
type Label = String
type CitationKey = String

data Document = Document
  { docTitle :: Maybe [Inline]
  , docAuthor :: Maybe [Inline]
  , docDate :: Maybe [Inline]
  , docBlocks :: [Block]
  }
  deriving (Show, Eq)

data Block
  = Paragraph [Inline]
  | Header Int [Inline] (Maybe Label)
  | UnorderedList [[Block]]
  | CodeBlock String
  deriving (Show, Eq)

data Inline
  = Plain String
  | Italic [Inline]
  | Bold [Inline]
  | Underline [Inline]
  | Strike [Inline]
  | Citation [CitationKey]
  | Reference [Inline] [Label]
  deriving (Show, Eq)
```

Several design choices in this representation deserve explicit comment.

The metadata fields on `Document` are typed as `Maybe [Inline]` rather than `Maybe String`. This allows a title or author field to carry the same nested formatting that a paragraph can, which is necessary because the test corpus includes formatted titles such as `My ~Complex~ ~*_Simple*_ Paper~`.

The `UnorderedList` constructor carries a list of lists of blocks. The outer list represents the items of the list, and the inner list represents the block-level content of a single item. This allows a list item to contain multiple paragraphs, a nested header, or another structural element.

The `Reference` constructor carries both a list of inline content and a list of labels. This supports references with descriptive text such as `[the features section](#sec:features)`, references without descriptive text such as `[](#sec:intro)`, and multi-target references that point to several labels at once.

The `Citation` constructor carries a list of citation keys, which lets a single citation in the source map onto a single `\cite` command with several comma-separated arguments. This matches the conventional way that LaTeX citations are written when several papers are referenced together.

8 Parsing Examples

The following examples illustrate how concrete `markTeX` input is transformed into AST nodes. They are taken directly from the test inputs used during development.

8.1 Header With Label

Listing 11: Input header with label.

```
## Introduction {#sec:intro}
```

Listing 12: Corresponding AST node.

```
Header 2 [Plain "Introduction"] (Just "sec:intro")
```

8.2 Reference to Multiple Labels

Listing 13: Input reference with multiple labels.

```
See [these sections](#sec:intro,sec:background, sec:list-header).
```

Listing 14: Simplified AST representation.

```
Paragraph
[ Plain "See "
, Reference [Plain "these sections"]
    ["sec:intro", "sec:background", "sec:list-header"]
, Plain "."
]
```

8.3 Citation With Multiple Keys

Listing 15: Input citation with two keys.

```
[@paper1, paper_2026]
```

Listing 16: Corresponding AST node.

```
Citation ["paper1", "paper_2026"]
```

9 Validation & Semantic Analysis

A document that parses successfully is not yet a document that can be rendered. The parser ensures that the input conforms to the grammar, but it does not ensure that the document is internally consistent. The validation stage closes this gap.

Validation is implemented in `MarkTex.Render.LaTeX` and runs in two passes. The first pass collects every label defined by a header into a `LabelTable`, which is a `Data.Map` keyed by label name. The second pass walks the document and confirms that every label referenced by a `Reference` node appears in the table.

9.1 Label Collection

The label collector traverses the document blocks and inserts each labeled header into the table. Encountering the same label twice is treated as error:

Listing 17: Label collection (from `MarkTex.Render.LaTeX`).

```
collectLabels :: Document -> Either String
  LabelTable
collectLabels doc = goBlocks M.empty (
  docBlocks doc)
  where
    goBlock table block =
      case block of
        Header _ _ (Just lbl) ->
          if M.member lbl table
            then Left ("Duplicate label: "
++ lbl)
            else Right (M.insert lbl block
table)
        ...
```

The function returns `Either String LabelTable`, so a duplicate-label error short-circuits the rest of the pipeline. The renderer is never invoked on a document that contains conflicting labels.

9.2 Reference Checking

After labels are collected, the validator walks the AST again and inspects every `Reference` node. Each label in the reference must appear in the label table. If any label is missing, the validator returns an error that identifies the undefined label by name:

Listing 18: Reference checking (from `MarkTex.Render.LaTeX`).

```
checkInline :: LabelTable -> Inline ->
  Either String ()
checkInline table inline =
  case inline of
    Reference xs lbls ->
      mapM_ (checkInline table) xs >>
      mapM_
        (\lbl ->
          if M.member lbl table
            then Right ()
            else Left ("Undefined
reference: " ++ lbl)
        )
      lbls
    ...
```

The validator also recurses into the de-

scriptive inline content of a reference, which means that a reference such as `[see *also* the appendix](#sec:appendix)` is validated both for the existence of `sec:appendix` and for the well-formedness of the nested **also** italic node.

9.3 Why Validation is Separated From Rendering

Keeping validation separate from rendering produces a simpler and more trustworthy renderer. The renderer assumes that every label exists, that every reference target is real, and that no two headers compete for the same label. Each of those assumptions corresponds to an invariant established by the preceding stage. The result is that the rendering code is structurally simple: it maps AST nodes onto strings without any defensive checking, and without any conditional logic that handles malformed inputs.

This division also matches the standard compiler convention of separating syntactic analysis, semantic analysis, and code generation into independent phases. Errors are caught at the earliest phase that can identify them, and later phases operate over progressively stronger guarantees.

10 LaTeX Generation

The renderer converts a validated AST into a complete LaTeX document. It emits the document preamble, the metadata commands, the body content, and a closing bibliography section. The preamble pulls in the packages needed by the constructs that `markTeX` supports, which currently consists of `hyperref`, `cleveref`, `geometry`, and `ulem`. The `ulem` package is required because the underline and strikethrough features use `\underline` and `\sout`.

10.1 Inline Rendering

Inline rendering is implemented as a recursive function that pattern-matches on the `Inline` constructor. Each case emits the LaTeX form corresponding to that constructor:

Listing 19: Inline rendering (from `MarkTex.Render.LaTeX`).

```
renderInline :: Inline -> String
renderInline inline =
  case inline of
    Plain s      -> escapeLatex s
    Italic xs    -> "\\textit{" ++
renderInlines xs ++ "}"
    Bold xs     -> "\\textbf{" ++
renderInlines xs ++ "}"
    Underline xs -> "\\underline{" ++
renderInlines xs ++ "}"
    Strike xs   -> "\\sout{" ++
renderInlines xs ++ "}"
    Citation keys -> "\\cite{" ++
intercalate "," keys ++ "}"
    Reference xs lbls -> ...
```

References are slightly more involved because the output depends on whether the reference has descriptive text and on how many labels it carries. A reference with text and one or more labels emits a `\hyperref` linked to the first label, followed by a parenthesized `\Cref` that names every label. A reference with no descriptive text emits only the `\Cref`. A reference with no labels emits the empty string, although this case is unreachable in practice because such a reference would have already failed validation.

10.2 Block Rendering

Block rendering follows the same case-by-case pattern. Headers are mapped onto the corresponding sectioning command according to nesting depth, with levels one through three producing `\section`, `\subsection`, and `\subsubsection`, and deeper levels falling back to `\paragraph` and `\subparagraph`. Unordered lists are rendered into an `itemize` environment. Paragraphs are rendered as their inline contents followed by a blank line. Code blocks are wrapped in a `verbatim` environment, which preserves their contents byte for byte.

10.3 Escaping

Because the target language is LaTeX, plain text must be escaped before it appears in the output.

The renderer applies a character-by-character escape function to every `Plain` node and to every label that appears in a `\label` or `\Cref` argument. The escape table handles all of the LaTeX-sensitive characters:

Listing 20: LaTeX escaping (from `MarkTex.Render.LaTeX`).

```
escapeLatex :: String -> String
escapeLatex = concatMap escapeChar
  where
    escapeChar c =
      case c of
        '#' -> "\\#"
        '$' -> "\\$"
        '%' -> "\\%"
        '&' -> "\\&"
        '_' -> "\\_"
        '{' -> "\\{"
        '}' -> "\\}"
        '[' -> "\\["
        ']' -> "\\]"
        '~' -> "\\textasciitilde{}"
        '^' -> "\\textasciicircum{}"
        '\\' -> "\\textbackslash{}"
        _ -> [c]
```

Correct escaping matters more than it might first appear. A single unescaped `%` introduces a LaTeX comment that swallows the remainder of a line, and a single unescaped `_` outside of math mode causes a compilation error. The escape function is therefore one of the few parts of the system that is exercised on every character of every paragraph.

11 End-to-End Example

The clearest demonstration of the system is the round-trip from source to generated LaTeX, which the project test document exercises in full. A condensed version of that test appears below. Complete version is attached at the end of the report.

11.1 markTeX Input

Listing 21: Sample markTeX input.

```
% Title: My ~~Complex~~ **_Simple*_ Paper
**
```



```
% Author: \~John\_Doe
% Date: 2026

# Introduction {#sec:intro}

This is ~~~~strike~~,
start bold with ~~~~strike~~ inside_ inside.

- First list item with References: See
  [](#sec:intro) and
  the [features section](#sec:features).
- [paper1, paper_2026] and [paper1]
  mixed with formatting.

# Background {#sec:background}

## Features {#sec:features}

'''
print("Hello, MarkTex!")
**not bold**
[@not_a_citation]
'''
```

11.2 Generated LaTeX

Two details in this output are worth pointing out. First, the contents of the code block are emitted verbatim, including the **not bold** sequence and the `[@not_a_citation]` sequence, neither of which were parsed as `markTeX` syntax. Second, the empty-text reference `[](#sec:intro)` is rendered as a bare `\Cref`, while the named reference `[features section](#sec:features)` is rendered as a `\hyperref` followed by a parenthesized `\Cref`. Both forms are useful in practice and are produced from the same AST constructor.

Listing 22: Generated LaTeX produced by `markTeX`.

```
\documentclass{article}
\usepackage{utf8}{inputenc}
\usepackage{hyperref}
\usepackage{cleveref}
\usepackage[margin=1in]{geometry}
\usepackage[normalem]{ulem}

\title{My \sout{Complex} \textbf{\underline{\textit{Simple}}} Paper}
```

```
\author{\textasciitilde{John\_Doe}}
\date{2026}

\begin{document}
\maketitle

\section{Introduction}\label{sec:intro}

This is \textit{~~~~strike~~},
\textbf{start bold with \textit{~~~~strike~~ inside_ inside}.}

\begin{itemize}
\item First list item with References: See
  \Cref{sec:intro} and the
  \hyperref[sec:features]{features section}
  (\Cref{sec:features}).
\item \cite{paper1,paper_2026} and \cite{paper1}
  mixed with formatting.
\end{itemize}

\section{Background}\label{sec:background}
\subsection{Features}\label{sec:features}

\begin{verbatim}
print("Hello, MarkTex!")
**not bold**
[@not_a_citation]
\end{verbatim}

\bibliographystyle{plain}
\bibliography{references}
\end{document}
```

12 Design Decisions

Several design decisions shaped the final implementation. Each is documented here with the trade-off it represents.

12.1 Choice of Haskell

Haskell was chosen because algebraic data types and pattern matching make both the AST and the multi-pass walks over the AST very compact. The `Inline` type has seven constructors, and the renderer handles all seven in roughly fifteen lines of code. A com-

parable implementation in a language without algebraic data types would either require visitor classes, nested conditionals, or an explicit tag field on every node.

12.2 Choice of Parsec

Parsec was selected over a separate parser generator for two reasons. First, the grammar of `markTeX` is small enough that the cost of a separate generator would outweigh the benefit. Second, parser combinators compose in the host language, which makes it easy to write helpers such as `labelParser` or `citationKey` and reuse them across multiple top-level parsers. The cost of this choice is that error messages from Parsec are less informative than those produced by a dedicated generator, which is noted as a limitation in Section 15.

12.3 Separating Blocks and Inline Elements

The grammar separates block-level constructs from inline constructs. Block-level parsers produce `Block` values and define document structure. Inline parsers produce `Inline` values and define formatting inside blocks. The two groups never overlap. This decision mirrors the conventional Markdown grammar and keeps the parser implementation manageable, because each parser can assume that it operates within a known context.

12.4 Recursive Inline Formatting

Inline formatting constructors carry a list of inline values rather than a flat string. This allows arbitrarily nested formatting to be represented without flattening or duplication. The alternative, which would have been to encode formatting as a stack of opening and closing tokens, was rejected because it would have pushed the responsibility for matching delimiters into the renderer and would have prevented the validator from reasoning about references that appear inside formatting.

12.5 Validation Before Rendering

The renderer assumes a valid document. Validation runs in a separate phase and returns `Either String` so that errors short-circuit the pipeline. This produces clearer error messages, simplifies the renderer, and matches standard compiler practice. The cost is a second pass over the AST, which is negligible for any realistic document size.

12.6 Verbatim Semantics for Code Blocks

Code blocks are not parsed as `markTeX` syntax. The contents are captured as a single string by `codeParser` using `manyTill anyChar (try (string "`"))` and emitted unchanged inside a LaTeX `verbatim` environment. This is the only place in the language where a block construct is opaque to the inline parsers, and it exists specifically to allow code examples to contain characters that would otherwise be interpreted as formatting.

13 Challenges

Several aspects of the implementation required noticeable effort.

The first was nested inline parsing. Bold and italic share the asterisk as a delimiter, and the standard approach of using `between` from Parsec does not handle the case where a bold construct contains an italic construct that ends just before the closing bold delimiter. The implementation addresses this by giving each formatting parser an explicit end-of-construct predicate that uses `notFollowedBy` and `lookAhead` to disambiguate `***` sequences.

The second was distinguishing syntax contexts. A code block must accept characters that the inline parser would otherwise treat as syntax. The implementation handles this by reading the entire content of a code block with `anyChar` until the closing triple backtick, which bypasses the inline parser entirely.

The third was reference validation. References can carry multiple labels, nest inside formatting, and contain inline formatting in their descriptive

text. The validator therefore must walk both the labels of a reference and the inline content of that reference, while simultaneously checking against a table populated by an earlier walk. The two-pass structure described in Section 9 keeps these concerns separated.

The fourth was LaTeX escaping. The escape function must handle every character that has special meaning in LaTeX, and it must do so without breaking the few characters that the AST itself introduces. In particular, the backslash, the curly braces, and the square brackets all appear in LaTeX commands emitted by the renderer, so the escape function is applied only to user-supplied content rather than to the generated wrapping.

14 Testing and Results

Testing was conducted using a single comprehensive test document that exercises every supported language feature in combination. The test document includes:

- A formatted title with nested strikethrough, bold, underline, and italic.
- Escaped LaTeX-sensitive characters in metadata and body content.
- All four inline formatting constructs in isolation and in combination.
- A bold paragraph containing italic, underline, and strikethrough children.
- References without descriptive text, references with descriptive text, and references that span multiple labels.
- Citations with one key and citations with multiple keys.
- An unordered list containing paragraphs, references, citations, and a nested subsection.
- A code block whose content includes sequences that would be parsed as formatting outside the code block.

- Two top-level sections with internal cross-references between them.

The full document compiles end-to-end through the `markTeX` pipeline and the resulting LaTeX compiles cleanly under `pdflatex` and `bibtex` without errors. The generated PDF preserves the nesting behavior described above, including the case in which a bold span contains italic content that contains underline content that contains strikethrough content.

Negative tests were also run informally. Introducing a duplicate label produces a `Duplicate label` error from the validator and prevents any LaTeX output from being written. Introducing a reference to a nonexistent label produces an `Undefined reference` error with the offending label name. Both errors are surfaced before `pdflatex` is invoked, which means that the user sees a clear semantic error rather than a cryptic LaTeX compilation error.

15 Limitations

The current implementation is deliberately bounded. The following features are not supported in the present version:

- Tables.
- Images and figure captions.
- Mathematical expressions and equation environments.
- Ordered lists.
- Block quotations.
- Footnotes.
- Inline code spans, as distinct from code blocks.
- Full Markdown compatibility, including reference-style links and setext-style headers.
- Configurable document classes, font sizes, or geometry options.
- Detailed parser error messages with line and column information.

The validator also has narrow scope. It checks duplicate labels and undefined references, but it does not check for unused labels, unreachable sections, or stylistic issues such as headings that skip levels. These would be straightforward additions but were considered outside the goal of producing a working compiler pipeline within the project timeline.

16 Future Work

Several extensions would build naturally on the existing implementation.

Adding tables would require a new block constructor and a corresponding parser that recognizes pipe-delimited rows. The renderer would emit a LaTeX `tabular` environment inside a `table` float, and the validator would need to confirm that each row contains the same number of columns.

Adding images would require a new inline or block constructor, a parser for a Markdown-style `![alt](path)` syntax, and a renderer that emits `\includegraphics` inside a `figure` environment. Optional captions and labels would extend the existing label-collection machinery without modification.

Adding mathematical expressions would require either a delimited inline construct or a dedicated block environment. The simplest path would be to treat content between dollar signs as opaque math that is passed through to LaTeX unchanged, in the same spirit as the existing code block treatment.

Improving parser error messages would benefit every author who uses the system. Parsec exposes source position information, and threading that information into the AST would make it possible to report the line and column of an undefined reference or a duplicate label.

Other natural directions include adding command-line flags for output paths, supporting multiple bibliography styles, generating outputs other than LaTeX, and adding an automated test suite that verifies both positive and negative cases programmatically.

17 Connection to Programming Language Principles

The project applies several ideas from the programming language curriculum to a problem domain outside of conventional programming languages. The language has a designed concrete syntax, a separate abstract syntax that captures structure independently of surface form, a parser that translates between the two, a semantic analysis pass that enforces invariants the parser cannot, and a code generator that emits a target language.

The concrete syntax was designed to balance familiarity with expressiveness. Authors who already know Markdown can use `markTeX` without learning much new syntax, while authors who need citations or labeled references can write them in a form that is shorter than the equivalent LaTeX.

The abstract syntax was designed to make later stages simple. Recursive inline formatting is encoded directly into the type structure. Optional metadata and optional labels are encoded with `Maybe`. References carry both descriptive content and a list of target labels in a single constructor.

The parser uses Parsec to build the grammar compositionally. Each construct has a small dedicated parser, and the parsers compose with the alternative combinator to form larger parsers. `Lookahead` and `notFollowedBy` are used sparingly to resolve genuine ambiguities, such as the boundary between italic and bold delimiters.

Semantic analysis runs as a two-pass walk over the AST, mirroring the distinction between collection and checking that is common in name-resolution and type-checking algorithms. The collection pass populates a symbol table indexed by label, and the checking pass reports any reference that does not resolve.

Code generation walks the validated AST and emits LaTeX strings. Each AST constructor maps onto a specific LaTeX form, and the renderer applies escaping at the point where user-supplied content enters the output. This matches the standard structure of code generators that produce concrete syntax in a target language from a tree-shaped intermediate representation.

18 Conclusion

`markTeX` implements a small Markdown-inspired compiler that translates a focused authoring language into LaTeX. The system is organized as a four-stage pipeline composed of parsing, AST construction, validation, and rendering, and it produces a final PDF through the standard LaTeX toolchain. The supported feature set covers metadata, headers, paragraphs, unordered lists, code blocks, recursive inline formatting, citations, labels, single-target references, and multi-target references. Documents containing duplicate labels or undefined references are rejected before any LaTeX output is generated, which keeps the renderer simple and gives the author clear semantic error messages.

The project demonstrates how a modest set of programming language ideas produces a usable tool when applied to the right problem domain. The authoring experience is closer to Markdown than to LaTeX, while the output is closer to LaTeX than to Markdown, and the compiler enforces structural correctness in between. Although the language is not a replacement for either Markdown or LaTeX in their full generality, it occupies a useful middle ground for the kind of short, structured technical writing that academic and engineering work routinely requires.